MediaSync 2.1 Practical Execution Plan

Prepared after re-checking the handover documents, local source code, deployed AWS services, live FastAPI OpenAPI schema, deployed Streamlit cluster app, and PostgreSQL database state on 10 July 2026.

Execution Principle

Tasos is correct: this should not be treated as a greenfield roadmap. The current system already contains two working applications and the shared database assets required for the MVP:

- App 1: Author Profile & Feedback System.
- App 2: Cluster Intelligence System.
- Shared PostgreSQL database with news articles, topic clusters, cluster members, scores, topics, source tables, Greek/English author-style corpora, users, assignments, evaluations, and generation history.

The practical objective is to connect existing pieces into one MediaSync 2.1 workflow:

Login -> Cluster Dashboard -> Cluster Detail -> Best Sources / RAG Context -> Author or Publisher Style Selection -> Draft Generation -> Editorial Review -> Feedback -> History.

Current Verified Baseline

Area	Verified status
App 1 URL	`https://tools-heath-occurrence-mambo.trycloudflare.com`
App 1 health/API docs	`/api/health` returns 200; `/docs` returns 200
App 2 URL	`https://curve-fundamentals-sympathy-moms.trycloudflare.com`
AWS host	`98.84.126.189`, hostname `ip-172-31-20-240`
Running services	FastAPI `5035`, Streamlit `8105`, PostgreSQL `5432`, vLLM Qwen2.5 `8000`, bge-m3 embeddings `8080`
Existing FastAPI paths	`/api/auth/*`, `/api/exploration/*`, `/api/generation/generate`, `/api/evaluation/*`, `/api/health`
Existing DB volume	`news_pool` ~213,690; `topic_clusters` ~119,571; `cluster_members` ~214,237; `greek_author_styles` ~142,935; `english_author_styles` ~89,723
Existing cluster data	Greek: 118,178 clusters; English: 1,316 clusters

Area	Verified status
Existing MMR data	`cluster_members.mmr_rank` populated for ~210,792 rows
Current gap found	App 2 handover says Top-5 MMR, but deployed `fetch_rag_articles()` currently orders by `LENGTH(np.content) DESC`; this is a refactor, not new R&D

1 to 1.5 Week Unified MVP Delivery Target

Based on the re-audit, the realistic target is not a full production rebuild. The realistic target is a focused unified MVP in approximately 1 to 1.5 weeks by reusing existing App 1, App 2, database, RAG, feedback, and generation assets.

This 1 to 1.5 week target applies only to MediaSync 2.1 unification and usability:

- No new ingestion.
- No new model decision.
- No production architecture discussion first.
- No rebuilding existing components.
- Production hardening starts after the unified flow is working.
- Included: one usable end-to-end flow from login to cluster selection, RAG context, author/publisher style generation, editorial review, feedback, and history.
- Excluded: new ingestion, full recommendation engine, full publisher intelligence, new Qwen3 AWS deployment, automated learning loop, and production hardening.

Practical Execution Table

Item	Already built	Needs integration / refactoring	Truly new development
Login / authentication	**Current status:** Built in App 1. JWT login, register, role routing, admin/user separation exist. **Exact evidence:** `App/backend/routers/auth.py`; deployed APIs `/api/auth/login`, `/api/auth/register`, `/api/auth/users`, `/api/auth/assignments`, `/api/auth/my-assignments`; tables `app_users`, `author_assignments`; React routes `/login`, `/admin`, `/user`.	**Missing:** App 2 has no login. Cluster workflow must be placed behind App 1 authentication inside the React/FastAPI app. **Owner:** MUBIII for app integration; Brainfood/MediaSync for final user-role rules. **Delivery time:** 1 day. **Visible output:** User logs in and lands inside unified MediaSync 2.1 app instead of separate App 1/App 2 entry points.	None for MVP. Existing authentication should be reused.

Item	Already built	Needs integration / refactoring	Truly new development
Cluster feed	**Current status:** Built in App 2 Streamlit. Cluster browsing works from `topic_clusters`, with filters/search and pinned demo clusters. **Exact evidence:** `/data/scripts/app_editorial.py` and local `Code/app_editorial.py`; functions `fetch_db_stats()`, `fetch_pinned()`, `fetch_clusters()`; tables `topic_clusters`, `cluster_scores`, `cluster_topics`.	**Missing:** Move cluster feed from Streamlit into React UI and expose through FastAPI. Keep same DB logic; do not rebuild clustering. **Owner:** MUBIII for React/FastAPI integration; Brainfood for MediaSync UI placement/acceptance. **Delivery time:** 2 days. **Visible output:** Logged-in editor sees Greek/English cluster feed inside unified MediaSync 2.1 UI.	None for MVP.
Cluster detail	**Current status:** Built in App 2. Cluster detail shows title, summary, category, score, dates, sources, source distribution, and article list. **Exact evidence:** `fetch_cluster()`, `fetch_sources()`, `fetch_articles()` in `app_editorial.py`; tables `topic_clusters`, `cluster_scores`, `cluster_sources`, `cluster_members`, `news_pool`.	**Missing:** Convert Streamlit detail view into React detail page and FastAPI endpoint. Preserve existing DB queries. **Owner:** MUBIII for integration; Brainfood for editorial fields expected in MediaSync detail view. **Delivery time:** 1-2 days. **Visible output:** Editor opens a cluster and sees detail, sources, and articles in the unified app.	None for MVP.
Cluster APIs	**Current status:** DB queries exist in App 2, but not as FastAPI endpoints. Current deployed OpenAPI has no `/api/clusters` route. **Exact evidence:** App 1 OpenAPI paths list auth/exploration/generation/evaluation only; App 2 has direct Streamlit DB functions.	**Missing:** Wrap existing App 2 queries as FastAPI endpoints: `/api/clusters`, `/api/clusters/{id}`, `/api/clusters/{id}/articles`, `/api/clusters/{id}/sources`. **Owner:** MUBIII. **Delivery time:** 1-2 days. **Visible output:** Cluster data available through authenticated JSON APIs and rendered by React.	None beyond API wrapping. This is integration/refactor, not new clustering.
RAG context builder	**Current status:** Built conceptually and partially implemented in App 2. It builds a source-article context and sends it to Qwen3/OpenRouter. **Exact evidence:** `generate_article(cluster, rag_articles)` in `app_editorial.py`; `RAG_TOP_N = 5`; tables `cluster_members`, `news_pool`.	**Missing:** Move RAG context creation into FastAPI and return context preview to React. Refactor selection to use `mmr_rank` instead of longest content. **Owner:** MUBIII. **Delivery time:** 1 day. **Visible output:** Editor can preview the exact 5 source articles that will be sent to the LLM before generation.	None for MVP.

Item	Already built	Needs integration / refactoring	Truly new development
MMR source selection	**Current status:** Data exists; DB has `cluster_members.mmr_rank` populated for ~210,792 rows. Handover describes Top-5 MMR. **Exact evidence:** table `cluster_members.mmr_rank` ; script `Code/06_kmeans_mmr.py` ; live DB count confirms MMR ranks exist.	**Missing:** Deployed App 2 currently orders RAG articles by `LENGTH(np.content) DESC`, not `mmr_rank` . Replace ordering with `cm.mmr_rank ASC NULLS LAST`, with fallback to similarity/content length. **Owner:** MUBIII. **Delivery time:** 0.5-1 day. **Visible output:** RAG preview shows Top-5 MMR-ranked articles from the selected cluster.	None. This is a correction/refactor of existing behavior.
Article generation from cluster	**Current status:** Built in App 2. Cluster-based generation exists using cluster title, summary, category, and selected source articles. **Exact evidence:** `generate_article()` in `app_editorial.py` ; Qwen3/OpenRouter call; source context assembled from `news_pool` .	**Missing:** Move generation call into FastAPI and invoke it from React cluster detail page. Store result in shared history. **Owner:** MUBIII for integration; Brainfood for acceptance of generated output format. **Delivery time:** 1-2 days. **Visible output:** Editor generates an article from a selected cluster inside unified MediaSync 2.1.	None for MVP. Existing generation should be reused.
Author style generation	**Current status:** Built in App 1. Author selection, author-style retrieval, and style-transfer generation exist. **Exact evidence:** `App/backend/routers/generation.py` ; `/api/generation/generate` ; tables `greek_author_styles` , `english_author_styles` ; frontend `GenerateContent.jsx` , `User.jsx` , `AuthorPanel.jsx` .	**Missing:** Connect selected cluster context into App 1 generation flow instead of free-text/web-search-only topic. Add author selector to cluster generation screen. **Owner:** MUBIII. **Delivery time:** 1-2 days. **Visible output:** Editor selects cluster + author and receives a cluster-grounded article in that author's style.	None for author style. Existing implementation is reusable.
Publisher style generation	**Current status:** Partially built at data/schema level but not complete as a user-facing generation mode. Publisher profiles exist but embeddings are not populated. Author-style corpora include publisher IDs. **Exact evidence:** table `publisher_profiles` has 5 rows; `publisher_profiles.embedding_vector` currently 0 populated; author-style tables include publisher metadata.	**Missing:** For MVP, implement lightweight publisher style selection using existing publisher/profile metadata or publisher-filtered style examples. Do not block unified flow on full publisher intelligence. **Owner:** MUBIII for MVP publisher-style option; Brainfood for publisher style rules and editorial acceptance. **Delivery time:** 1-2 days for simple MVP; deeper publisher intelligence after MVP. **Visible output:** Editor can choose publisher style when author style is not selected.	Minimal new UI/API option for publisher-style mode. Full semantic publisher intelligence is not required for the 1 to 1.5 week unified MVP.

Item	Already built	Needs integration / refactoring	Truly new development
Feedback	**Current status:** Built in App 1 for author-generation evaluation pairs. App 2 has no feedback loop. **Exact evidence:** `/api/evaluation/submit`; `App/backend/routers/evaluation.py`; `evaluations` table; React `User.jsx`, `EvaluationsPanel.jsx`.	**Missing:** Attach feedback form to cluster-generated draft and include `cluster_id`, source article IDs, and style mode in saved payload/history. Existing `evaluations` schema may need small extension or use `generation_history` + `evaluations`. **Owner:** MUBIII for integration; Brainfood for required feedback fields. **Delivery time:** 1 day. **Visible output:** Editor reviews generated draft, edits/approves/rejects, and feedback is saved.	Small schema/API adjustment may be needed to link feedback to cluster IDs.
Evaluation	**Current status:** Built in App 1. hTER and chrF are computed; COMET placeholder exists. Only 2 live evaluation rows exist. **Exact evidence:** `metrics_service.py`; `/api/evaluation/submit`, `/api/evaluation/list`, `/api/evaluation/stats`, `/api/evaluation/my`; table `evaluations`.	**Missing:** Use the same evaluation mechanism for cluster-generated drafts. Do not add new metrics for MVP. **Owner:** MUBIII. **Delivery time:** 0.5-1 day. **Visible output:** Admin/user can view evaluation records for unified cluster-author generation.	None for MVP. COMET can remain deferred.
Generation history	**Current status:** Table exists but is barely used. Live DB has 1 row. App 1 generation endpoint currently returns result but does not persist every generation. **Exact evidence:** table `generation_history`; live count = 1; `generation.py` has no insert into `generation_history`.	**Missing:** Persist every unified generation with user ID, cluster ID, author/publisher style, model, source article IDs, generated title/content, prompt version, and timestamps. **Owner:** MUBIII. **Delivery time:** 1 day. **Visible output:** History tab shows all generated drafts from the unified flow.	Small persistence/API addition.
Recommendation engine	**Current status:** Not production-built. Some ingredients exist: publisher profiles, cluster scores, categories, languages, dates, source counts. **Exact evidence:** `publisher_profiles` table has 5 rows; `cluster_scores`, `topic_clusters`, `cluster_sources` populated; publisher embeddings not populated.	**Missing:** For the 1 to 1.5 week MVP, do not build a full recommendation engine. Use a simple “recommended feed” view based on existing score/category/language/source count and optional publisher filter. Brainfood must define editorial recommendation rules later. **Owner:** Brainfood owns editorial/product rules; MUBIII can implement simple MVP ranking once rules are confirmed. **Delivery time:** MVP simple ranking 1 day; full engine after MVP. **Visible output:** Optional “Recommended” filter/tab using existing scores, not a full personalization engine.	Simple ranking tab if needed. Full recommendation engine is truly new and should be post-MVP unless Brainfood insists.

Item	Already built	Needs integration / refactoring	Truly new development
Editorial intelligence	**Current status:** Partially built. Cluster scores, topics, source distribution, article counts, recency, and author style analysis exist. Structured facts/entities/angles are not populated. **Exact evidence:** populated `cluster_scores`, `cluster_topics`, `cluster_sources`; empty `cluster_facts`, `cluster_entities`, `cluster_angles`; App 1 `exploration.py`; App 2 cluster detail UI.	**Missing:** In MVP, expose existing intelligence only: scores, topics, sources, source diversity, selected RAG articles, author style signals. Do not create new fact/entity extraction first. **Owner:** MUBIII for display integration; Brainfood/MediaSync for editorial taxonomy expectations. **Delivery time:** 1 day as part of cluster detail. **Visible output:** Cluster detail page shows editorial intelligence already available in DB.	Structured facts/entities/angles extraction is truly new/post-MVP.
Publisher intelligence	**Current status:** Minimal/partial. Publisher profiles table exists; publisher IDs exist in corpora; source/domain counts exist. Publisher embeddings are empty. **Exact evidence:** `publisher_profiles` = 5 rows; `publisher_profiles_with_embedding` = 0; `cluster_sources` populated; author style tables include publisher IDs.	**Missing:** For MVP, use publisher as a filter/style selector only. Defer full publisher intelligence until after unified flow. **Owner:** Brainfood defines publisher strategy; MUBIII implements selected UI/API behavior. **Delivery time:** 1 day for simple selector/filter; deeper intelligence post-MVP. **Visible output:** Publisher selector/filter appears in unified flow where relevant.	Full publisher embeddings, publisher similarity, and publisher-specific recommendation logic are truly new/post-MVP.
Learning loop	**Current status:** Partially built through feedback/evaluation table. No active automated learning loop. **Exact evidence:** `evaluations` table has 2 rows; `editor_corrections` table exists but 0 rows; handover describes feedback as future LoRA data.	**Missing:** For MVP, save feedback/corrections consistently. Do not implement training loop now. **Owner:** MUBIII for data capture; Brainfood decides later use for training/fine-tuning. **Delivery time:** Included in feedback/history work. **Visible output:** Every generated draft has saved feedback/correction data.	Actual learning/fine-tuning loop is truly new/post-MVP.
Inference setup	**Current status:** Built and running. AWS hosts Qwen2.5-7B via vLLM and bge-m3 embeddings. App 2 uses Qwen3 through external OpenRouter path. **Exact evidence:** EC2 processes: `vllm serve --model Qwen/Qwen2.5-7B-Instruct`; `text-embeddings-router --model-id BAAI/bge-m3`; ports `8000`, `8080`.	**Missing:** For MVP, standardize which existing endpoint the unified app calls. Do not make a new model decision or new deployment first. **Owner:** MUBIII for config wiring; Brainfood approves which existing endpoint is acceptable for demo. **Delivery time:** 0.5 day. **Visible output:** Unified generation works using existing inference path.	None for MVP. New AWS Qwen3 deployment is post-MVP.

Item	Already built	Needs integration / refactoring	Truly new development
Production deployment	**Current status:** Basic deployment exists. App 1 and App 2 are live through Cloudflare tunnels; FastAPI, Streamlit, PostgreSQL, vLLM, embeddings are running. Root disk remains a risk at ~94% used. **Exact evidence:** EC2 live services verified; URLs return 200; `/dev/root` 94% used; `/data` healthy.	**Missing:** For the 1 to 1.5 week MVP, deploy unified app on existing server and expose through current mechanism. Production hardening should follow after unified flow works. **Owner:** MUBIII for deploying unified app; Brainfood/Alex for final MediaSync deployment route and production hosting standards. **Delivery time:** 1 day for MVP deployment; hardening after MVP. **Visible output:** One URL showing unified MediaSync 2.1 flow.	Stable production domain, monitoring, backup policy, Secrets Manager, disk cleanup, and permanent infra are post-MVP hardening.

Proposed 1 to 1.5 Week Execution Sequence

Day window	Output
Day 1	Add authenticated cluster APIs in FastAPI by wrapping existing App 2 SQL logic.
Day 2	Add React cluster dashboard inside the existing App 1 authenticated UI.
Day 3	Add cluster detail page, source inspection, and article list inside React.
Day 4	Refactor RAG source selection to use existing `mmr_rank`; show Top-5 RAG context preview.
Day 5	Connect selected cluster context to existing author-style generation flow.
Day 6	Add editorial review and generation-history persistence for cluster-generated drafts.
Day 7	Connect existing feedback/evaluation flow to cluster-generated drafts.
Days 8-9	Integrate MediaSync 2.1 navigation/branding, deploy one unified URL, QA, fixes, and demo script.

What Can Be Demonstrated At The End Of 1 to 1.5 Weeks

The expected result after 1 to 1.5 weeks is not a fully polished production system. The expected result is a unified MediaSync 2.1 MVP where the already-built App 1 and App 2 capabilities are brought under one authenticated product umbrella and can be demonstrated as one connected editorial workflow.

The success criterion is workflow unification: a user should be able to move through the complete editorial path without switching between separate applications.

The 1 to 1.5 week goal does not guarantee perfect generation quality, perfect author-style matching, perfect publisher-style behavior, or perfect context awareness. The goal is to reuse the current generation, style retrieval, cluster RAG, feedback, and history components inside one unified workflow. Therefore, the generation quality expected during this phase should be understood as the current system's existing output quality, now accessible through a connected MediaSync 2.1 flow.

- One MediaSync 2.1 URL.
- Login with existing users.
- Cluster feed inside authenticated app.
- Cluster detail with sources, scores, topics, and articles.
- Top-5 RAG context preview using existing MMR ranks.
- Generate article from a selected cluster.
- Apply selected author style.
- Optional simple publisher-style mode, if Brainfood accepts lightweight MVP behavior.
- Editorial review and correction submission.
- Evaluation/history visible in the app.

Expected limitations at this stage:

- Some UI polish may still be pending.
- Publisher style may be lightweight MVP behavior, not full publisher intelligence.
- Recommendation may be a simple score/filter-based view, not a full personalization engine.
- Generation quality will depend on the existing model endpoint and existing source data.
- Production hardening, monitoring, permanent domain setup, and infrastructure cleanup remain post-MVP.

Explicitly Deferred Until After Unified MVP

- New ingestion pipelines.
- New model hosting decision.
- Qwen3 AWS deployment.
- Full recommendation engine.
- Full publisher intelligence.
- Structured fact/entity/angle extraction.
- Automated learning/fine-tuning loop.
- Production hardening beyond what is necessary to expose the unified demo.

Main Correction From Previous Roadmap

The previous roadmap over-separated several already-built capabilities into standalone milestones. The revised execution plan treats them as existing assets and focuses only on integration/refactoring required to turn App 1 + App 2 + shared database into one usable MediaSync 2.1 workflow.